GitHub User Search / Weather App (API + Cache)

Problem: Build a search UI that calls a public API as the user types (GitHub user search; a weather-by-city app is the same pattern), shows results, and opens a detail view. It must handle debouncing, out-of-order responses, request cancellation with `AbortController`, and an in-memory cache.

Tech stack: React (Vite, JavaScript) · **Time limit:** 60 minutes (default, none given) · **Often asked at:** Startups, Walmart

---

PART A: Thought process

The same steps work for any machine coding task:

1. **Clarify requirements.** Separate must-haves from nice-to-haves, and ask before assuming.
2. **List interactions.** Every action the user can take and what should happen. This becomes your test checklist.
3. **Sketch the architecture.** Components, hooks, services, and who talks to whom.
4. **Design state.** Store the minimum and derive the rest. Decide who owns each piece.
5. **Define data and contracts.** API shapes, example payloads, error formats.
6. **Trace one flow end to end.** Keystroke → state → side effect → UI.
7. **Spot the hard parts early.** Here: race conditions, cancellation, cache, rate limits.
8. **Build in layers.** First a version that works, then add one layer at a time.
9. **Cover every state.** Idle, loading, success, empty, error, stale.
10. **Wrap up.** Edge cases, trade-offs, what you would add with more time.

Time budget (60 min)

Phase	Minutes
Requirements + clarifying questions	5
Interactions, architecture, state design	8
Basic fetch + list rendering	10
Debounce + AbortController + cache	15
Detail view, error/empty/loading states	12
Accessibility, polish, edge cases	5
Wrap-up and trade-offs	5

Think out loud. The interviewer scores your reasoning as much as your code. Say why you debounce, why you abort, why the cache lives outside React. If you get stuck, say what you are trying, so they can nudge you.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- A search input. Typing searches GitHub users by username.
- Show up to 10 matches with avatar and login.
- Clicking a result opens a detail card (name, bio, repos, followers, following, location, GitHub link).
- Repeat searches and repeat detail views come from the cache, with no network call.
- Show clear loading, empty, and error states, with a Retry button on errors.

Non-functional requirements

Concern	What we do
Performance	Debounce (400 ms) so typing `gaearon` sends 1 request, not 7. Cache responses.
Correctness	Never show results for an old query (race conditions). Abort stale requests.
Scalability	A cache with a size cap (LRU, 50 entries) and a TTL (5 min), so memory stays bounded.
Error handling	Readable messages for rate limit (403/429), 404, 422 and network failures. Retry.
Accessibility	Labelled input, `role="status"` live region for result counts, `role="alert"` for errors, real `<button>`s for results, visible focus.
Reusability	`useFetch(url)` is generic. The detail view uses it, and so would a weather app.
Extensibility	URL builders live in one service file. Changing the API means changing one file.

Clarifying questions

1. Search as you type, or only on Enter / a Search button? *(Assume as you type, debounced.)*
2. How fresh must data be? Can a cached result be a few minutes old? *(Assume a 5-minute TTL is fine.)*
3. Do we need pagination or infinite scroll of results? *(Assume the top 10 only.)*
4. Is there an auth token? Unauthenticated GitHub search allows about 10 requests/minute. *(Assume no token, so handle rate limits well.)*
5. Should the cache survive a page reload (localStorage)? *(Assume memory only.)*
6. Keyboard navigation of results (arrow keys)? *(Assume Tab + Enter on buttons is enough.)*

Assumptions we build with

- Search fires 400 ms after the user stops typing, and only for 2+ characters.
- The query is trimmed and lowercased, so `"Dan "` and `"dan"` share one cache entry.
- In-memory cache, keyed by full URL, 5-minute TTL, at most 50 entries (LRU eviction).
- While a new search loads, the old list stays visible but dimmed (no blank flash).
- No auth token. Rate-limit errors show the time the limit resets.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	Empty input, status "Search GitHub users."
Type 1 character	No request. Status "Type at least 2 characters."
Type `gaearon` quickly	Exactly one request, 400 ms after the last key.
Request in flight	Status "Searching…" (or the old list dimmed if one exists).
Results arrive	List of users; status "6 users found."
No matches	Status `No users found for "zzqxw".`
Type `react`, then quickly `vue`	The `react` request is aborted; only `vue` results can show.
Search `gaearon` again later	Shown instantly from cache, no network call.
Click a user	Detail card loads; the selected row is highlighted.
Click a user seen before	Detail comes from cache instantly.
Clear the input	List disappears, back to idle. No request.
Rate limited (403)	"GitHub rate limit reached. Try again at 10:04 PM." + Retry.
Network down	"Failed to fetch"-style message + Retry.
Click Retry	Same URL fetched again (errors are never cached).
Component unmounts mid-request	Request aborted; no state update on an unmounted component.

3. Design the architecture

Module	Responsibility
`App`	Owns the query and the selected user. Builds the search URL. Renders status.
`UserList`	Renders result buttons. Knows nothing about fetching.
`UserDetail`	Fetches and shows one user by login.
`useDebounce(value, delay)`	Returns the value only after it stops changing for `delay` ms.
`useFetch(url)`	Cache check → fetch with AbortController → `{status, data, error, retry}`.
`services/cache.js`	`createCache({ttl, max})`: LRU + TTL map, plain JS, no React.
`services/githubApi.js`	URL builders + `fetchJson` that turns HTTP errors into messages.

```
App  (query, selected)
 ├─ useDebounce(query) ──> term ──> searchUsersUrl(term)
 ├─ useFetch(url) ──────────────> responseCache ── fetchJson ── GitHub API
 ├─ UserList (users, selected, onSelect)
```

```
└ UserDetail (login)
     └ useFetch(userUrl(login)) -> same responseCache
```

Start simple: everything in `App` with one `useEffect`. Pull out `useFetch` once the detail view needs the same logic. Pull out the cache when you want to test it alone.

4. Design the state / data model

State or field	Purpose
`query` (App)	Raw input value. Controlled input, updates on every key.
`selected` (App)	Login of the user whose detail card is open.
`status` (useFetch)	`'idle' | 'loading' | 'success' | 'error'`.
`data` (useFetch)	Last successful response (kept while the next one loads).
`error` (useFetch)	Readable message for the error state.
`attempt` (useFetch)	Counter. Bumping it re-runs the effect (Retry).
`responseCache` (module)	URL → `{value, expiresAt}`. Lives outside React, shared by all hooks.

```
// App
const [query, setQuery] = useState('');
const [selected, setSelected] = useState(null);

// useFetch
const [state, setState] = useState({ status: 'idle', data: null, error: null });
const [attempt, setAttempt] = useState(0);
```

Senior notes

- **Derived, not stored:** `term` (debounced + normalised), `url`, `users = data?.items ?? []`, and the status message are all computed during render. Storing them would give two sources of truth.
- **No `isLoading` + `isError` booleans.** One `status` field makes impossible combinations (loading *and* error) impossible.
- **The cache is not React state.** Writing to it should not re-render anything, and it must outlive the components that read it.
- **Ownership:** `App` owns what the user chose (query, selection). The hook owns what the network returned.

5. Design the data structures and contracts

Search users: `GET https://api.github.com/search/users?q=gaearon&per_page=10`

```
{
  "total_count": 6,
```

```
  "incomplete_results": false,
  "items": [
    { "id": 810438, "login": "gaearon", "avatar_url": "https://avatars.githubusercontent.com/u/810438?
v=4", "html_url": "https://github.com/gaearon" }
  ]
}
```

`id` is the stable React key. Logins can change on GitHub, but ids never do.

User detail: `GET https://api.github.com/users/gaearon`

```
{ "login": "gaearon", "name": "dan", "bio": null, "public_repos": 301, "followers": 93937,
  "following": 174, "location": "London", "avatar_url": "...", "html_url": "https://github.com/gaearon"
}
```

Error contract. `fetchJson` always throws an `Error` with a message ready for the UI, or rethrows `AbortError` unchanged:

HTTP	Message
403/429 + `x-ratelimit-remaining: 0`	`GitHub rate limit reached. Try again at <reset time>.`
404	`Not found.`
422	`Invalid search query.`
other	`Request failed (<status>).`
network failure	browser `TypeError` message

Cache contract. `createCache({ ttl, max, now })` returns `{ get(key), set(key, value), size(), clear() }`. `get` returns `undefined` for missing or expired keys.

6. Trace the data flow

User types `gaearon`, then clicks the first result:

```
keystrokes g-a-e-a-r-o-n
   │  setQuery() on each key → re-render; input stays responsive
   ▼
useDebounce: each key clears the previous timer
   │  400 ms of silence → debounced = "gaearon"
   ▼
render: term = "gaearon" → url = /search/users?q=gaearon&per_page=10
   ▼
useFetch effect (url changed)
   │  previous effect's cleanup → controller.abort() on the old request
   │  responseCache.get(url) → miss
   │  setState(loading, keep old data)
```

```
   │  fetchJson(url, controller.signal)
   ▼
response 200 → responseCache.set(url, data) → setState(success)
   ▼
App renders UserList + "6 users found." (announced by the live region)
   ▼
click "gaearon" → setSelected("gaearon") → UserDetail mounts
   │  useFetch(/users/gaearon) → miss → fetch → cache → card renders
```

7. Important technical challenges

Challenge 1: Race conditions (out-of-order responses)

Problem. The user types `react`, waits, then types `reactive`. The `react` request is slow (2 s); `reactive` is fast (200 ms). Without protection, `reactive` results show first, then `react` results arrive and overwrite them. The input says `reactive`, the list shows `react`.

Solution. Every effect run owns its own `AbortController`. When `url` changes, React runs the old effect's cleanup first, which aborts the old request. An aborted fetch rejects with `AbortError`, which we ignore. So an old response can never reach `setState`.

```
useEffect(() => {
  const controller = new AbortController();
  fetchJson(url, controller.signal)
    .then((data) => setState({ status: 'success', data, error: null }))
    .catch((err) => {
      if (err.name === 'AbortError') return; // a newer request replaced this one
      setState({ status: 'error', data: null, error: err.message });
    });
  return () => controller.abort();
}, [url]);
```

Abort also saves bandwidth and rate-limit budget. A plain `let ignore = true` flag in the cleanup fixes the UI bug too, but the request still runs to completion.

Challenge 2: Debouncing without stale closures

Problem. Calling the API on every `onChange` sends 7 requests for `gaearon`, which burns GitHub's 10/minute limit after one word.

Solution. Debounce the *value*, not the handler. The input stays controlled and instant; only the derived `term` waits. Because the timer is reset in the effect cleanup, there are no stale closures and no `useRef` timer juggling.

```
export function useDebounce(value, delay = 400) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
```

```
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);
  return debounced;
}
```

Debounce and abort solve different problems. Debounce cuts requests while the user is typing. Abort handles a request that is already in flight when the query changes again.

Challenge 3: A bounded cache (TTL + LRU)

Problem. A plain object cache grows forever during a long session, and it serves week-old data.

Solution. A `Map` keeps insertion order, so its first key is the least recently used. On `get`, delete and re-insert to mark the entry as fresh. On `set`, evict the first key when over `max`. Each entry stores `expiresAt`, and expired entries are dropped when read. `now` is injectable, so tests don't need real timers.

```
function get(key) {
  const entry = store.get(key);
  if (!entry) return undefined;
  if (now() > entry.expiresAt) { store.delete(key); return undefined; }
  store.delete(key); store.set(key, entry); // mark as most recently used
  return entry.value;
}
function set(key, value) {
  store.delete(key);
  store.set(key, { value, expiresAt: now() + ttl });
  if (store.size > max) store.delete(store.keys().next().value);
}
```

Only successful responses are cached. An error is never cached, so Retry really retries.

Challenge 4: Turning HTTP failures into useful UI

Problem. `fetch` only rejects on network failure. A 403 rate-limit response resolves normally, and `res.json()` then gives `{ message: "API rate limit exceeded" }`. The UI would try to read `.items` of that and show "No users found", which is wrong.

Solution. Check `res.ok` in one place and throw a readable error. Read GitHub's rate-limit headers to tell the user *when* they can try again.

```
if (res.ok) return res.json();
if ((res.status === 403 || res.status === 429) && res.headers.get('x-ratelimit-remaining') === '0') {
  const reset = Number(res.headers.get('x-ratelimit-reset')) * 1000;
  throw new Error(`GitHub rate limit reached. Try again at ${new Date(reset).toLocaleTimeString()}.`);
}
```

8. Implement in the correct order

#	Step	Working result
1	Vite app, controlled `<input>` with a label	Typing updates state
2	`useEffect` that fetches the search URL on every change, renders logins	Ugly but working search
3	`fetchJson` with `res.ok` check and error messages	Errors no longer look like "no results"
4	One `status` field: idle / loading / success / error	Loading and error UI
5	`useDebounce` + min 2 characters + trim/lowercase	1 request per word
6	`AbortController` in the effect cleanup, ignore `AbortError`	No race conditions
7	Extract `useFetch(url)` with `retry`	Reusable hook, Retry button
8	`createCache` (TTL + LRU) used inside `useFetch`	Repeat searches are instant
9	`UserList` with avatars, `UserDetail` using the same hook	Full feature
10	Keep old data while loading (dimmed), empty-state message	No flicker
11	Live region, `role="alert"`, focus styles, responsive grid	Accessible + mobile
12	Node self-check for cache and `fetchJson`	Logic verified

9. Cover all states and edge cases

State / case	What the user sees
Initial (empty input)	"Search GitHub users." No request.
1 character	"Type at least 2 characters." No request.
Only spaces	Treated as empty (trimmed).
Loading, no previous list	"Searching…"
Loading, previous list exists	Old list dimmed with `aria-busy`, until new data arrives
Success	List + "N users found." (with thousands separator)
Empty	`No users found for "term".`
Error: rate limit	Reset time + Retry
Error: offline / DNS	Browser message + Retry
Error: 422 (bad query)	"Invalid search query."
Cache hit	Instant success, no loading state
Cache entry expired	Treated as a miss, refetched
Fast typing / query change mid-request	Old request aborted, old response ignored

State / case	What the user sees
Same query with different case or spaces	Same cache key, no extra request
Special characters (`a&b` , `c#`)	`encodeURIComponent` keeps the URL valid
Selected user from an old search	Detail card stays open; it fetches by login, not by list
Detail data from the previous user while loading	Guarded with `data.login !== login` → "Loading …"
Unmount during fetch	Aborted in cleanup, no setState warning
StrictMode double effect in dev	First request aborted, second wins (shows abort works)
User has no name / bio / location	Falls back to login; missing fields hidden

10. Final structure

```
src/
├── main.jsx
├── App.jsx                query, selection, status message
├── App.css
├── components/
│   ├── UserList.jsx       result buttons
│   └── UserDetail.jsx     one user's card
├── hooks/
│   ├── useDebounce.js
│   └── useFetch.js        cache + AbortController + retry
└── services/
    ├── cache.js           createCache({ ttl, max })
    └── githubApi.js       URL builders + fetchJson
selfcheck.mjs              node assert tests for cache + fetchJson
```

11. Full working code

`src/services/cache.js`

```
// In-memory LRU cache with a time-to-live per entry.
// Map keeps insertion order, so the first key is always the least recently used.
export function createCache({ ttl = 5 * 60 * 1000, max = 50, now = Date.now } = {}) {
  const store = new Map();

  function get(key) {
    const entry = store.get(key);
    if (!entry) return undefined;
    if (now() > entry.expiresAt) {
      store.delete(key);
      return undefined;
    }
```

```
    // re-insert to mark as most recently used
    store.delete(key);
    store.set(key, entry);
    return entry.value;
  }

  function set(key, value) {
    store.delete(key);
    store.set(key, { value, expiresAt: now() + ttl });
    if (store.size > max) store.delete(store.keys().next().value);
  }

  return { get, set, size: () => store.size, clear: () => store.clear() };
}
```

src/services/githubApi.js

```
const BASE = 'https://api.github.com';

export const searchUsersUrl = (query) =>
  `${BASE}/search/users?q=${encodeURIComponent(query)}&per_page=10`;

export const userUrl = (login) => `${BASE}/users/${encodeURIComponent(login)}`;

// Turns HTTP failures into readable messages; AbortError is passed through untouched.
export async function fetchJson(url, signal, fetchImpl = fetch) {
  const res = await fetchImpl(url, { signal, headers: { Accept: 'application/vnd.github+json' } });
  if (res.ok) return res.json();

  if ((res.status === 403 || res.status === 429) && res.headers.get('x-ratelimit-remaining') === '0') {
    const reset = Number(res.headers.get('x-ratelimit-reset')) * 1000;
    const at = reset ? new Date(reset).toLocaleTimeString() : 'later';
    throw new Error(`GitHub rate limit reached. Try again at ${at}.`);
  }
  if (res.status === 404) throw new Error('Not found.');
  if (res.status === 422) throw new Error('Invalid search query.');
  throw new Error(`Request failed (${res.status}).`);
}
```

src/hooks/useDebounce.js

```
import { useEffect, useState } from 'react';

export function useDebounce(value, delay = 400) {
  const [debounced, setDebounced] = useState(value);
```

```
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);

  return debounced;
}
```

src/hooks/useFetch.js

```
import { useCallback, useEffect, useState } from 'react';
import { createCache } from '../services/cache';
import { fetchJson } from '../services/githubApi';

// One cache shared by every useFetch call, keyed by URL.
export const responseCache = createCache({ ttl: 5 * 60 * 1000, max: 50 });

const IDLE = { status: 'idle', data: null, error: null };

// url = null means "nothing to fetch" (idle).
export function useFetch(url) {
  const [state, setState] = useState(IDLE);
  const [attempt, setAttempt] = useState(0);

  useEffect(() => {
    if (!url) {
      setState(IDLE);
      return;
    }

    const cached = responseCache.get(url);
    if (cached) {
      setState({ status: 'success', data: cached, error: null });
      return;
    }

    const controller = new AbortController();
    // keep the previous data on screen while the new request is in flight
    setState((prev) => ({ ...prev, status: 'loading', error: null }));

    fetchJson(url, controller.signal)
      .then((data) => {
        responseCache.set(url, data);
        setState({ status: 'success', data, error: null });
      })
      .catch((err) => {
        if (err.name === 'AbortError') return; // a newer request replaced this one
```

```
        setState({ status: 'error', data: null, error: err.message });
      });

    // cleanup runs when url changes or the component unmounts -> cancels the stale request
    return () => controller.abort();
  }, [url, attempt]);

  const retry = useCallback(() => setAttempt((n) => n + 1), []);

  return { ...state, retry };
}
```

src/components/UserList.jsx

```
export default function UserList({ users, selected, onSelect, stale }) {
  return (
    <ul className={`user-list${stale ? ' stale' : ''}`} aria-busy={stale}>
      {users.map((user) => (
        <li key={user.id}>
          <button
            type="button"
            className={user.login === selected ? 'active' : ''}
            aria-pressed={user.login === selected}
            onClick={() => onSelect(user.login)}
          >
            <img src={user.avatar_url} alt="" width="32" height="32" loading="lazy" />
            <span>{user.login}</span>
          </button>
        </li>
      ))}
    </ul>
  );
}
```

src/components/UserDetail.jsx

```
import { useFetch } from '../hooks/useFetch';
import { userUrl } from '../services/githubApi';

export default function UserDetail({ login }) {
  const { status, data, error, retry } = useFetch(userUrl(login));

  if (status === 'error') {
    return (
      <div className="detail" role="alert">
        <p>{error}</p>
```

```
        <button type="button" onClick={retry}>Retry</button>
      </div>
    );
  }
  // data may still belong to the previously selected user while loading
  if (!data || data.login !== login) return <div className="detail">Loading {login}…</div>;

  return (
    <article className="detail">
      <img src={data.avatar_url} alt={`${data.login} avatar`} width="96" height="96" />
      <h2>{data.name || data.login}</h2>
      {data.bio && <p>{data.bio}</p>}
      <dl>
        <dt>Repos</dt><dd>{data.public_repos}</dd>
        <dt>Followers</dt><dd>{data.followers}</dd>
        <dt>Following</dt><dd>{data.following}</dd>
        {data.location && (<><dt>Location</dt><dd>{data.location}</dd></>)}
      </dl>
      <a href={data.html_url} target="_blank" rel="noreferrer">View on GitHub</a>
    </article>
  );
}
```

src/App.jsx

```
import { useState } from 'react';
import { useDebounce } from './hooks/useDebounce';
import { useFetch } from './hooks/useFetch';
import { searchUsersUrl } from './services/githubApi';
import UserList from './components/UserList';
import UserDetail from './components/UserDetail';
import './App.css';

const MIN_CHARS = 2;

export default function App() {
  const [query, setQuery] = useState('');
  const [selected, setSelected] = useState(null);

  // normalise so "Dan" and "dan " share one cache entry
  const term = useDebounce(query, 400).trim().toLowerCase();
  const url = term.length >= MIN_CHARS ? searchUsersUrl(term) : null;
  const { status, data, error, retry } = useFetch(url);
  const users = data?.items ?? [];

  let message = '';
  if (!url) message = query.trim() ? `Type at least ${MIN_CHARS} characters.` : 'Search GitHub users.';
```

```
  else if (status === 'loading' && !users.length) message = 'Searching…';
  else if (status === 'success' && !users.length) message = `No users found for "${term}".`;
  else if (status === 'success') message = `${data.total_count.toLocaleString()} users found.`;

  return (
    <main className="app">
      <h1>GitHub User Search</h1>
      <label htmlFor="q">Username</label>
      <input
        id="q"
        type="search"
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="e.g. gaearon"
        autoComplete="off"
      />

      <p className="status" role="status" aria-live="polite">{message}</p>

      {status === 'error' && (
        <div className="error" role="alert">
          <p>{error}</p>
          <button type="button" onClick={retry}>Retry</button>
        </div>
      )}

      <div className="layout">
        {url && users.length > 0 && (
          <UserList users={users} selected={selected} onSelect={setSelected} stale={status ===
'loading'} />
        )}
        {selected && <UserDetail login={selected} />}
      </div>
    </main>
  );
}
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
```

```
  </StrictMode>,
);
```

src/App.css

```
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; background: #f6f8fa; color: #1f2328; }

.app { max-width: 820px; margin: 0 auto; padding: 24px 16px; }
label { display: block; font-weight: 600; margin-bottom: 4px; }
input { width: 100%; padding: 10px 12px; font-size: 16px; border: 1px solid #d0d7de; border-radius:
6px; }
input:focus-visible, button:focus-visible { outline: 2px solid #0969da; outline-offset: 2px; }

.status { min-height: 1.5em; color: #57606a; }
.error { background: #ffebe9; border: 1px solid #ff8182; border-radius: 6px; padding: 8px 12px; }

.layout { display: grid; grid-template-columns: 1fr; gap: 16px; }
@media (min-width: 640px) { .layout { grid-template-columns: 1fr 1fr; } }

.user-list { list-style: none; margin: 0; padding: 0; transition: opacity 0.2s; }
.user-list.stale { opacity: 0.5; }
.user-list button {
  display: flex; align-items: center; gap: 10px; width: 100%; padding: 8px;
  background: #fff; border: 1px solid #d0d7de; border-radius: 6px; margin-bottom: 6px;
  cursor: pointer; font-size: 15px; text-align: left;
}
.user-list button.active { border-color: #0969da; background: #ddf4ff; }
.user-list img { border-radius: 50%; }

.detail { background: #fff; border: 1px solid #d0d7de; border-radius: 6px; padding: 16px; align-self:
start; }
.detail img { border-radius: 50%; }
.detail dl { display: grid; grid-template-columns: auto 1fr; gap: 4px 12px; }
.detail dt { color: #57606a; }
.detail dd { margin: 0; }
```

selfcheck.mjs (run with `node selfcheck.mjs`)

```
import assert from 'node:assert/strict';
import { createCache } from './src/services/cache.js';
import { fetchJson, searchUsersUrl } from './src/services/githubApi.js';

// cache: TTL expiry
let t = 0;
const c = createCache({ ttl: 100, max: 2, now: () => t });
```

```
c.set('a', 1);
assert.equal(c.get('a'), 1);
t = 101;
assert.equal(c.get('a'), undefined, 'expired entry is dropped');

// cache: LRU eviction (reading "a" makes "b" the oldest)
t = 0;
c.set('a', 1); c.set('b', 2); c.get('a'); c.set('c', 3);
assert.equal(c.get('b'), undefined, 'least recently used is evicted');
assert.equal(c.get('a'), 1);
assert.equal(c.size(), 2);

// url building escapes user input
assert.ok(searchUsersUrl('a b&c').includes('q=a%20b%26c'));

// fetchJson: success, 404, rate limit, abort
const res = (status, body, headers = {}) => ({
  ok: status < 300, status, json: async () => body, headers: { get: (k) => headers[k] ?? null },
});
assert.deepEqual(await fetchJson('u', null, async () => res(200, { x: 1 })), { x: 1 });
await assert.rejects(fetchJson('u', null, async () => res(404)), /Not found/);
await assert.rejects(
  fetchJson('u', null, async () => res(403, null, { 'x-ratelimit-remaining': '0', 'x-ratelimit-reset':
'1700000000' })),
  /rate limit/,
);
const ctrl = new AbortController();
const slow = (url, { signal }) => new Promise((_, reject) =>
  signal.addEventListener('abort', () => reject(new DOMException('Aborted', 'AbortError'))));
const pending = fetchJson('u', ctrl.signal, slow);
ctrl.abort();
await assert.rejects(pending, { name: 'AbortError' });

console.log('selfcheck passed');
```

Weather app variant. Keep the hooks, the cache and the components' structure. Only `githubApi.js` changes, for example with Open-Meteo (no key needed):

```
export const geocodeUrl = (city) =>
  `https://geocoding-api.open-meteo.com/v1/search?name=${encodeURIComponent(city)}&count=5`;
export const forecastUrl = ({ latitude, longitude }) =>
  `https://api.open-meteo.com/v1/forecast?
latitude=${latitude}&longitude=${longitude}&current=temperature_2m,weather_code`;
```

City search → `useFetch(geocodeUrl(term))`, pick a city → `useFetch(forecastUrl(city))`. Use a shorter TTL (about 10 minutes) because weather changes.

12. Wrap-up

Edge cases handled. Debounced typing, minimum length, normalised cache keys, aborted stale requests, ignored `AbortError`, rate-limit message with reset time, 404/422, offline, errors not cached, Retry, stale list kept while loading, detail guarded against the previous user's data, unmount cleanup, URL encoding.

Trade-offs to mention

- **Data cache vs promise cache.** We cache resolved data. Two components asking for the same URL at the same moment will both fetch. Caching the in-flight promise dedupes them, but then one component's abort must not cancel the other's request.
- **Hand-rolled vs library.** TanStack Query / SWR give dedupe, stale-while-revalidate, background refetch and devtools. In an interview, building it yourself shows you understand it. In production, use the library.
- **Module-level cache.** Simple and shared, but global. It survives across tests unless cleared (`responseCache.clear()`), and it would leak between users in SSR.
- **400 ms debounce.** Lower feels snappier but costs rate-limit budget. Higher saves requests but feels laggy.
- **Keep old data while loading.** No flicker, but the user briefly sees results that don't match the input. We dim them to make that clear.

With more time

- Unit tests with Vitest + React Testing Library + MSW (mock the API, assert one request per word and no out-of-order render).
- Arrow-key navigation of results (combobox pattern with `aria-activedescendant`).
- Stale-while-revalidate: show cached data, refetch in the background.
- Persist the cache to `sessionStorage`; sync the query to the URL (`?q=gaearon`) so it can be shared.
- Pagination with the `Link` header, an optional token for higher rate limits, retry with exponential backoff.

13. Counter questions the interviewer may ask

Requirements

Q1. Why search as you type instead of a Search button?

It's the expected UX for lookups. But with a tight rate limit (10/min for GitHub search), a submit button is a fair choice. Ask the interviewer. The code barely changes: replace the debounced value with the value at submit time.

Q2. Why a minimum of 2 characters?

One letter matches millions of users, the result is useless, and it costs a request. The minimum is a product decision. Ask about it, and keep it in a constant.

State design

Q3. Why one `status` field instead of `isLoading` and `isError`?

Separate booleans allow impossible states like loading and error at the same time, and every branch has to check both. A single enum makes the UI a simple switch.

Q4. Why isn't the cache in `useState` or Context?

Writing to the cache should not re-render anything, and the cache must outlive any component. A module-level object does both. Context would be useful only if different parts of the tree needed different caches, for example per user in SSR.

Q5. Why keep the old `data` while loading?

Clearing it makes the list blink empty on every keystroke pause. Keeping it (dimmed, `aria-busy`) feels faster. The trade-off is a brief mismatch between input and list, which the dimming signals.

Race conditions and abort

Q6. Is debounce enough to prevent race conditions?

No. Debounce lowers the number of requests, but if the user pauses for 400 ms, types again and pauses again, two requests can still be in flight. Only abort (or an ignore flag) makes sure that the latest query wins.

Q7. What if the API doesn't support cancellation?

`abort()` works on any `fetch`. The server may still do the work, but the browser drops the response. Without fetch (an SDK, a promise you can't cancel), use a flag: `let active = true; return () => { active = false }`, and check `active` before `setState`. Or compare a request id stored in a ref.

Q8. Why does the network tab show two `/users/gaearon` requests in dev?

React StrictMode mounts, unmounts and remounts effects in development. The first request is aborted by the cleanup, and the second one wins. It's a free test that the cleanup is correct, and it does not happen in production.

Q9. What happens if the user clicks Retry while a request is still in flight?

`attempt` changes, so the effect re-runs. Its cleanup aborts the current request first, then a fresh one starts. Retry is never doubled.

Cache

Q10. Why key the cache by URL?

The URL already contains everything that decides the response (endpoint, query, page size). It's unique and free. Normalising the query before building the URL makes `Dan` and `dan` share one entry.

Q11. How does your LRU work without a linked list?

JavaScript `Map` iterates in insertion order. On read, delete and re-insert the key, so it moves to the end. The first key is always the least recently used, so eviction is `store.keys().next().value`. Both are O(1).

Q12. Two components request the same URL at the same time. What happens?

Both miss the cache and both fetch. To dedupe, store the in-flight promise in a second map and return it to the second caller. Remove it when it settles. Then don't let one caller's abort cancel the shared request: give the shared fetch its own controller and abort it only when no caller is waiting.

Q13. How would you invalidate the cache?

TTL handles freshness. For explicit invalidation: `cache.delete(url)` after a mutation, `cache.clear()` on logout, or a version prefix in the key. Stale-while-revalidate (show cached data, refetch quietly) is the next step up.

Errors and API

Q14. `fetch` resolved but the UI showed "no users". Why?

`fetch` only rejects on network errors. A 403 or 500 resolves, and its JSON has no `items`. Always check `res.ok` and throw, as `fetchJson` does.

Q15. What if the API is slow, say 5 seconds?

The old list stays dimmed and the live region says "Searching…". Add a timeout with `AbortSignal.timeout(8000)` combined with the user signal (`AbortSignal.any([...])`), and show "Taking longer than usual" after a few seconds. Abort still protects against older slow responses.

Q16. How would you handle the rate limit better?

Show the reset time (done). Disable searching until then, raise the debounce, use an auth token (5000/hour on the REST API, and 30/minute for search), or proxy through your own backend that caches across users.

Performance and scale

Q17. What if a search could return 1 million results?

Never fetch them all. Use server pagination (`page`, `per_page`, the `Link` header) with "Load more" or infinite scroll, and virtualise the list if thousands of rows are rendered. Our cache key already includes the page parameters.

Q18. Should `UserList` be wrapped in `React.memo`?

Not at 10 rows: the cost of comparing props is about the same as rendering. Measure with the Profiler first. If the list grew large, memo plus a stable `onSelect` (state setters are already stable) would help.

Trade-offs / why not X

Q19. Why not TanStack Query or SWR?

In production, yes: they give dedupe, retries, background refetch, cache GC and devtools. In an interview, the point is to show you know what they do inside. Our `useFetch` is a small version of `useQuery`.

Q20. Why not debounce the `onChange` handler with lodash?

A debounced handler needs `useMemo` / `useRef` to stay stable across renders, needs cancelling on unmount, and makes the input feel laggy if you debounce `setQuery`. Debouncing the value keeps the input instant and the cleanup automatic.

Q21. How would you test this?

Pure logic (cache, `fetchJson`) with plain asserts or Vitest and an injected clock and fetch, as in `selfcheck.mjs`. UI with React Testing Library + MSW: fake timers to prove one request per word, a delayed MSW handler to prove that an old slow response never shows, and a 403 handler for the error UI.

Q22. How does this change for the weather app?

Only the service file: a geocoding URL for city search, then a forecast URL by coordinates. Same hooks, same cache (with a shorter TTL), plus unit formatting (°C/°F) and maybe browser geolocation for "my location".